# Тема №7. Решение оптимизационных задач с ограничениями

## 7.1 Краткие теоретические сведения

Оптимизационные задачи с ограничениями принадлежат к классу задач линейного программирования. Линейное программирование изучает методы нахождения экстремума линейной функции нескольких переменных при дополнительных ограничениях в виде линейных равенств (или неравенств), накладываемых на переменные. К данному классу задач относятся транспортные задачи, распределительные задачи, задачи календарного планирования и т.п.

Математическая модель задачи линейного программирования включает в себя систему линейных ограничений (равенств или неравенств) отражающих условие задачи, а также линейную функцию выражающую цель задачи. Такая функция называется целевой функцией.

В общем виде задача линейного программирования может быть сформулирована следующим образом: найти минимум (максимум) линейной функции [11, 13]

$$z = c_0 + c_1 x_1 + c_2 x_2 + ... + c_n x_n, \tag{7.1}$$

при условии, что переменные $x_1, x_2, ..., x_n$ не отрицательные и удовлетворяют системе ограничений

$$\begin{cases} a_{11} x_1 + a_{12} x_2 + ... + a_{1n} x_n \geq b_1, \\ a_{21} x_1 + a_{22} x_2 + ... + a_{2n} x_n \geq b_2, \\ ... \\ a_{m1} x_1 + a_{m2} x_2 + ... + a_{mn} x_n \geq b_m. \end{cases} \tag{7.2}$$

Точка, являющаяся решением задачи линейного программирования, располагается на границе области допустимых

решений. Эта область представляет собой многогранник, образованный неравенствами системы ограничений (7.2).

Одним из наиболее популярных методов решения задач линейного программирования является симплекс-метод. Идея симплекс-метода заключается в переборе вершин многогранника области допустимых решений. Данный перебор производится при условии последовательного улучшения целевой функции (7.1). В результате находится вершина многогранника, в которой целевая функция достигает максимума (или минимума).

## 7.2 Практическая работа

Цель работы: получить навыки решения задач линейного программирования с помощью средств и методов языка Python и библиотеки SciPy.

### 7.2.1 Пример выполнения работы

Решим с применением возможностей языка Python транспортную задачу.

Пусть имеются поставщики товаров с трех складов расположенных в разных городах. Объемы продукции на этих складах равны: $a_1 = 74$, $a_2 = 40$, $a_3 = 36$. В других трех городах находятся потребители, потребности которых в этих товарах равны соответственно $b_1 = 20$, $b_2 = 45$, $b_3 = 30$. Стоимости доставки $c_{ij}$ от *i*-го поставщика к *j*-му потребителю представлены матрицей

$$C = \begin{pmatrix} 7 & 3 & 6 \\ 4 & 8 & 2 \\ 1 & 5 & 9 \end{pmatrix}.$$

Требуется составить такой план перевозок товара от поставщиков к потребителям, чтобы стоимость его перевозки была минимальной.

Если $x_{ij}$ – количество перевозимого товара от *i*-го поставщика к *j*-му потребителю, то целевая функция для этой задачи примет вид

$$F = c_{11}x_{11} + c_{12}x_{12} + c_{13}x_{13} + ... = \sum_{i,j=1}^{3} c_{ij}x_{ij} \to \min .$$

Обозначим коэффициенты $c_{11} = c_1$, $c_{12} = c_2$, …, $c_{33} = c_9$, а переменные $x_{11} = x_1$, $x_{12} = x_2$, …, $x_{33} = x_9$. Тогда для целевой функции получим

$$F = c_1x_1 + c_2x_2 + c_3x_3 + ... = \sum_{i=1}^{9} c_ix_i \to \min .$$

Запишем ограничения задачи. Т.к. от каждого поставщика возможно поставить товара не больше, чем у него имеется, то

$$\begin{cases} x_1 + x_2 + x_3 \le 74, \\ x_4 + x_5 + x_6 \le 40, \\ x_7 + x_8 + x_9 \le 36. \end{cases}$$

При этом необходимо удовлетворить потребности каждого потребителя, т.о.

$$\begin{cases} x_1 + x_4 + x_7 = 20, \\ x_2 + x_5 + x_8 = 45, \\ x_3 + x_6 + x_9 = 30. \end{cases}$$

Тогда система ограничений задачи примет следующий вид:

$$\begin{cases} x_1 + x_2 + x_3 \le 74, \\ x_4 + x_5 + x_6 \le 40, \\ x_7 + x_8 + x_9 \le 36, \\ x_1 + x_4 + x_7 = 20, \\ x_2 + x_5 + x_8 = 45, \\ x_3 + x_6 + x_9 = 30, \\ x_i \ge 0, i = 1, ..., 9. \end{cases}$$

а целевая функция:

$$F = 7x_1 + 3x_2 + 6x_3 + 4x_4 + 8x_5 + 2x_6 + x_7 + 5x_8 + 9x_9 \to \min .$$

Для решения задач линейного программирования в Python существует несколько библиотек: PuLP, CVXOPT, SciPy.

Мы будем использовать модуль optimize из библиотеки SciPy [14]. Эту библиотеку отличает большее быстродействие и удобство ввода данных.

Из модуля optimize библиотеки SciPy импортируем функцию `linprog()`:

```
from scipy.optimize import linprog
```

Функция `linprog()` предназначена для решения задач линейного программирования и имеет вид:

```
scipy.optimize.linprog(c, A_ub=None, b_ub=None, A_eq=None,
b_eq=None, method='highs', etc.)
```

Её основными атрибутами являются: `c` – список коэффициентов целевой функции, `A_ub` – матрица коэффициентов условий-неравенств, `b_ub` – значения из правой части условий-неравенств, `A_eq` – матрица коэффициентов условий-равенств, `b_eq` – значения из правой части условий-равенств, `method` – метод решения задачи оптимизации.

Сначала мы создаем эти списки пустыми, а затем заполняем путем ввода значений с клавиатуры.

Для списка коэффициентов целевой функции вводим

```
c = []
c.append([float(i) for i in input('Введите коэффициенты'
         + 'целевой функции через запятую:').split(',')])
```

Команда append() добавляет к пустому списку список значений коэффициентов, которые мы вводим через запятую.

Функция split(',') разбивает строку, введенную с клавиатуры, на части по разделителю, указанному в скобках, и создает список символов из полученных частей.

Далее, создаем и заполняем матрицу коэффициентов условий-неравенств. Т.к. эта матрица двумерная, заполняем ее путем добавления в цикле строк к первоначальному пустому списку:

```
A_ub = []
N_ner = int(input('Количество ограничений-неравенств = '))
for j in range(N_ner):
    coeff_a_ub = input('Введите коэффициенты ' + str(j+1)
           + '-го неравенства через запятую: ').split(',')
    A_ub.append([float(i) for i in coeff_a_ub])
```

Аналогично заполняем остальные списки и матрицы:

```
b_ub = []
coeff_b_ub = input('Коэффициенты правой части '
    + 'ограничений-неравенств через запятую: ').split(',')
b_ub.append([float(i) for i in coeff_b_ub])

A_eq = []
N_rav = int(input('Количество ограничений-равенств= '))
for j in range(N_rav):
    coeff_a_eq = input('Введите коэффициенты ' + str(j+1)
             + '-го равенства через запятую: ').split(',')
    A_eq.append([float(i) for i in coeff_a_eq])

b_eq=[]
```

```
coeff_b_eq = input('Коэффициенты правой части '
      + 'ограничений-равенств через запятую: ').split(',')
b_eq.append([float(i) for i in coeff_b_eq])

print(linprog(c, A_ub, b_ub, A_eq, b_eq))
```

Особенностью функции `linprog()` является то, что для каждого уравнения системы ограничений требуется вводить коэффициенты всех переменных, входящих в целевую функцию. Если в уравнении системы ограничений не входит какая-либо переменная, то для неё вводится коэффициент равный нулю. Т.е. для рассматриваемой задачи система ограничений, коэффициенты которой будут вводиться в матрицы `A_ub` и `A_eq`, примет вид:

$$\begin{cases} x_1 + x_2 + x_3 + 0x_4 + 0x_5 + 0x_6 + 0x_7 + 0x_8 + 0x_9 \le 74, \\ 0x_1 + 0x_2 + 0x_3 + x_4 + x_5 + x_6 + 0x_7 + 0x_8 + 0x_9 \le 40, \\ 0x_1 + 0x_2 + 0x_3 + 0x_4 + 0x_5 + 0x_6 + x_7 + x_8 + x_9 \le 36, \\ x_1 + 0x_2 + 0x_3 + x_4 + 0x_5 + 0x_6 + x_7 + 0x_8 + 0x_9 = 20, \\ 0x_1 + x_2 + 0x_3 + 0x_4 + x_5 + 0x_6 + 0x_7 + x_8 + 0x_9 = 45, \\ 0x_1 + 0x_2 + x_3 + 0x_4 + 0x_5 + x_6 + 0x_7 + 0x_8 + x_9 = 30, \\ x_i \ge 0, i = 1, ..., 9. \end{cases}$$

Вводя коэффициенты этой системы ограничений, в результате работы кода программы получим:

```
con: array([8.03400049e-08, 1.98486838e-07,
            1.27598781e-07])
fun: 214.99999944270834
message: 'Optimization terminated successfully.'
nit: 5
slack: array([29.00000022, 10.00000009, 16.0000001 ])
status: 0
success: True
x: array([1.17814759e-08, 4.49999998e+01, 8.01637322e-09,
4.06707740e-08,       9.61431590e-09,       2.99999999e+01,
1.99999999e+01, 3.01544961e-08, 5.79416775e-09])
```

Итогом работы программы является набор данных, содержащий следующую основную информацию: `x` – вектор решения (значения переменных, при которых достигается цель задачи); `fun` – оптимальное значение целевой функции; `slack` – остатки в условиях-неравенствах (в рассматриваемой задаче это остатки товара на складах).

### 7.2.2 Задания для самостоятельной работы

В данной работе требуется решить задачу линейного программирования средствами языка Python.

В ходе выполнения работы требуется:

а) сформулировать словесный алгоритм программы;

б) построить блок-схему программы;

в) написать код программы;

г) сделать выводы и составить отчет.

Условие задачи для всех вариантов одинаковое: найти оптимальный план перевозок груза с трех складов в четыре пункта назначения. Конкретные значения запасов, потребностей и тарифов на перевозки представлены в индивидуальных вариантах.

Индивидуальные варианты задания:

1. Запасы на складах: 38 т, 50 т, 15 т. Потребности в пунктах назначения: 15 т, 25 т, 10 т, 8 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 7 & 9 & 3 \\ 5 & 11 & 8 & 6 \\ 6 & 10 & 12 & 3 \end{pmatrix}.$$

2. Запасы на складах: 48 т, 30 т, 15 т. Потребности в пунктах назначения: 10 т, 25 т, 10 т, 18 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 8 & 7 & 6 & 3 \\ 15 & 11 & 8 & 6 \\ 6 & 10 & 16 & 3 \end{pmatrix}.$$

3. Запасы на складах: 38 т, 20 т, 45 т. Потребности в пунктах назначения: 10 т, 25 т, 10 т, 18 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 8 & 7 & 7 & 3 \\ 5 & 11 & 18 & 6 \\ 6 & 10 & 12 & 3 \end{pmatrix}.$$

4. Запасы на складах: 28 т, 57 т, 15 т. Потребности в пунктах назначения: 25 т, 25 т, 10 т, 8 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 7 & 9 & 3 \\ 15 & 11 & 8 & 6 \\ 6 & 10 & 12 & 13 \end{pmatrix}.$$

5. Запасы на складах: 36 т, 50 т, 25 т. Потребности в пунктах назначения: 17 т, 20 т, 10 т, 18 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 7 & 9 & 13 \\ 5 & 11 & 8 & 6 \\ 16 & 10 & 12 & 3 \end{pmatrix}.$$

6. Запасы на складах: 38 т, 47 т, 10 т. Потребности в пунктах назначения: 15 т, 25 т, 10 т, 28 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 7 & 9 & 3 \\ 5 & 7 & 18 & 6 \\ 6 & 10 & 12 & 3 \end{pmatrix}.$$

7. Запасы на складах: 38 т, 50 т, 15 т. Потребности в пунктах назначения: 12 т, 15 т, 10 т, 28 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 6 & 7 & 9 & 3 \\ 5 & 11 & 8 & 11 \\ 8 & 10 & 12 & 3 \end{pmatrix}.$$

8. Запасы на складах: 48 т, 54 т, 10 т. Потребности в пунктах назначения: 25 т, 30 т, 10 т, 18 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 12 & 9 & 3 \\ 5 & 9 & 8 & 6 \\ 6 & 10 & 8 & 3 \end{pmatrix}.$$

9. Запасы на складах: 38 т, 50 т, 35 т. Потребности в пунктах назначения: 15 т, 25 т, 10 т, 11 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 7 & 9 & 13 \\ 15 & 11 & 8 & 6 \\ 6 & 8 & 12 & 3 \end{pmatrix}.$$

10. Запасы на складах: 58 т, 30 т, 15 т. Потребности в пунктах назначения: 5 т, 25 т, 10 т, 18 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 3 & 7 & 9 & 3 \\ 5 & 11 & 18 & 6 \\ 5 & 10 & 12 & 3 \end{pmatrix}.$$

11. Запасы на складах: 38 т, 50 т, 22 т. Потребности в пунктах назначения: 3 т, 15 т, 20 т, 8 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 7 & 5 & 3 \\ 3 & 11 & 8 & 6 \\ 6 & 10 & 8 & 8 \end{pmatrix}.$$

12. Запасы на складах: 18 т, 50 т, 35 т. Потребности в пунктах назначения: 9 т, 25 т, 9 т, 8 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 7 & 9 & 3 \\ 15 & 11 & 8 & 16 \\ 6 & 10 & 12 & 3 \end{pmatrix}.$$

13. Запасы на складах: 38 т, 44 т, 18 т. Потребности в пунктах назначения: 16 т, 25 т, 20 т, 8 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 10 & 4 & 9 & 3 \\ 5 & 11 & 8 & 6 \\ 6 & 9 & 9 & 3 \end{pmatrix}.$$

14. Запасы на складах: 38 т, 50 т, 35 т. Потребности в пунктах назначения: 15 т, 12 т, 10 т, 28 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 4 & 7 & 9 & 3 \\ 5 & 6 & 8 & 8 \\ 6 & 10 & 12 & 3 \end{pmatrix}.$$

15. Запасы на складах: 28 т, 48 т, 15 т. Потребности в пунктах назначения: 15 т, 15 т, 10 т, 8 т. Тарифы на перевозки (тыс. руб.):

$$C = \begin{pmatrix} 7 & 17 & 9 & 3 \\ 5 & 11 & 8 & 6 \\ 6 & 8 & 12 & 3 \end{pmatrix}.$$

## 7.3 Контрольные вопросы

1. Записать математическую формулировку задачи линейного программирования в общем виде.
2. Что такое область допустимых решений?
3. Как формируется область допустимых решений?
4. Как решается задача линейного программирования симплекс-методом?
5. Что такое целевая функция?
6. Геометрический смысл симплекс-метода.
7. Какое решение называется допустимым?
8. Записать математическую формулировку транспортной задачи.
9. Какой многогранник называется выпуклым?